

Short-answer key

1. **Why BCEWithLogitsLoss ?**

It combines the sigmoid and BCE in a numerically stable way. Applying `sigmoid` first can hurt stability and is redundant.

2. **Why use a very negative score for masked tokens?**

Softmax turns larger scores into larger weights. Padding tokens should get essentially zero weight, so you push them toward $-\infty$, not $+\infty$.

3. **Why is `dim=0` wrong?**

`dim=0` normalizes across the batch, so tokens from different examples compete with one another. Attention should normalize across the token dimension for each sample independently.

4. **Why does `zero_grad()` matter?**

PyTorch accumulates gradients by default. Without resetting, each minibatch adds onto the previous gradient, which changes the effective step size and usually destabilizes training.

5. **How to detect noisy labels?**

Look for examples with persistently high loss or high-confidence disagreement, like `prob≈0.99` with label `0` or `prob≈0.02` with label `1`. Then compare against nearest neighbors, a simpler baseline, repeated runs, or manual inspection if possible.

6. **What metric besides accuracy?**

If classes are imbalanced, report **F1**, **precision/recall**, or **PR-AUC**. If you care about ranking quality across thresholds, **AUROC** is also useful.

Why this is a good OpenAI-style mock

This mock is intentionally biased toward **debugging a model that runs but does not learn correctly**, not toward algorithm puzzles. That is the most consistent public signal: OpenAI's official interview guide emphasizes practical skills-based assessments and strong engineering quality; the public research-role summary emphasizes ML coding and ML debugging in PyTorch/NumPy; and the residency anecdote adds short-answer math on top.

One more prep note: OpenAI's official guide explicitly recommends reading the Charter, recent research/blog posts relevant to your team, and technical material like the *Deep Learning Book* and *Spinning Up in Deep RL*. Also, the public research-role summary says the recruiter's short description of each interview is often very informative, so treat that blurb like a syllabus.

A strong next step would be doing a second mock that is **NumPy-only**, with manual backprop, vectorized attention, and probability/statistics questions.